dfnWorks → GEOS Workbench

From zero on Windows — copy, paste, run

You do **not** need to know Git, Docker or Python. Follow the steps in order, paste one grey block at a time into PowerShell, press **Enter**, and wait for it to finish before the next block.

Total time: about **1½ hours** — roughly 20 minutes of downloads plus one 45–60 minute GEOS compile that runs unattended.

What you are installing

Program	Why	Where
Git	downloads (clones) the project code	https://git-scm.com/download/win
Python 3.10+	runs the panel	https://python.org/downloads
Docker Desktop	runs dfnWorks and the GEOS build without installing Linux tools	https://docker.com/products/docker-desktop

Plus three folders of code, downloaded for you in steps 3 and 4: this project, dfnWorks, and the GEOS solver source.

> **What you do NOT need:** no C++ compiler, no Linux, no GPU — and nothing to install by hand beyond the three programs above. The GEOS solver is not a program you download: with `-WithGeos` the setup script clones the GEOS source for you and compiles it *inside* a Docker container. GEOS' own `thirdPartyLibs` come pre-built inside the toolchain image and are used only during that container build — they never touch your PC.

Fast track — one command (skip to here if you like)

Install the three programs in step 1, make sure Docker Desktop is running, then paste this into PowerShell:

```
git clone https://github.com/sadafiiman/dfn-geos-workbench.git C:\dfn-project\dfn-geos-workbench
cd C:\dfn-project\dfn-geos-workbench
powershell -ExecutionPolicy Bypass -File .\setup_windows.ps1 -WithGeos
```

If you downloaded the folder instead of cloning it, open PowerShell in that folder and paste just the last line. (`setup_windows.cmd -WithGeos`, typed in a terminal, does the same — the `.cmd` passes arguments through — but double-clicking the file runs it *without* the flag and would skip GEOS, so the PowerShell line is the safe one.)

This one command does the rest of the tutorial for you:

1. checks Git, Python and Docker — and starts Docker Desktop for you if it is installed but not running;
2. downloads dfnWorks and this project into `C:\dfn-project` (step 3);
3. installs `numpy` and pulls the dfnWorks image, about 1.5 GB (step 4);
4. clones the GEOS source and **builds the GEOS solver image** `dfn-geos:1.1` — the long step: about **45–60 minutes** and about **8 GB** of disk, mostly silent (this is step 4b);
5. writes the settings (step 6);
6. starts the panel and opens the browser (step 5).

When it prints **Setup finished — all three stages are available**, jump to step 7 and make a case.

- It is safe to run again: if `dfn-geos:1.1` already exists the build is skipped, and the whole script then finishes in about a minute.

- Without `-WithGeos` you get everything *except* GEOS solving: stages 1–2 work, stage *3 · Run GEOS* is skipped. You can add GEOS any time later by running the same command with the flag.
- Next time, run `start_windows.cmd` (or `python run_panel.py`).

If you prefer to see each step happen, ignore this box and continue with step 1.

Step 1 — Install the three programs

1. **Git** — run the installer, keep every default, click *Next* to the end.
2. **Python** — on the first installer screen **tick “Add python.exe to PATH”*, *then *Install Now*.
3. **Docker Desktop** — install, accept the WSL2 prompt, restart the PC when asked. After restart, open **Docker Desktop** from the Start menu and wait until it says **Engine running** (whale icon in the tray, not animated).

> Docker Desktop must be running every time you use the panel.

Step 2 — Open PowerShell

Press **Win + X** → **Terminal** (or *Windows PowerShell*).

A window opens with a line ending in `>`. That is where you paste. To paste, right-click inside the window (or `Ctrl+V`), then press **Enter**.

Check the three programs answered correctly:

```
git --version
python --version
docker --version
```

Each must print a version number. If one says *"not recognized"*, that program did not install — see **Troubleshooting** at the end.

Step 3 — Download the project code

Two repositories are needed. Paste this whole block once:

```
mkdir C:\dfn-project
cd C:\dfn-project
git clone https://github.com/lanl/dfnWorks.git
git clone https://github.com/sadafiiman/dfn-geos-workbench.git
```

- `dfnWorks` — the fracture-network generator (used by the panel).
- `dfn-geos-workbench` — the panel, converters and report (this project).

> Use a short path like `C:\dfn-project`. Do **not** put it on the Desktop or in OneDrive: long paths and syncing break the mesh tools.

Step 4 — Install the Python package and the dfnWorks image

```
cd C:\dfn-project\dfn-geos-workbench
python -m pip install --upgrade pip
python -m pip install numpy
docker pull ees16/dfnworks:latest
```

The last line downloads about **1.5 GB**. Wait until it prints *"Status: Downloaded newer image"* or *"Image is up to date"*.

4b — Build the GEOS solver image (the long step: ~45–60 min)

```
powershell -ExecutionPolicy Bypass -File .\setup_windows.ps1 -WithGeos -NoStart
```

This is the same setup script the fast track uses. It re-checks everything above in seconds (*already cloned*, *numpy already present*, *image up to date*) and then spends its time on the real work:

1. clones the GEOS source into `C:\dfn-project\GEOS`;
2. pulls the `geosx/ubuntu24.04-gcc13` toolchain image — it already contains every third-party library GEOS needs;
3. downloads the GEOS submodules — several GB, quiet for a while;
4. compiles GEOS inside that container — **30–60 minutes with no output**; that is normal, it is a ~3000-file C++ build;
5. packages the result as a Docker image named `dfn-geos:1.1`.

Expect the whole step to take **45–60 minutes** and need about **8 GB** of free disk. It also writes the panel settings for you (step 6). The `-NoStart` switch only means *don't open the panel yet* — step 5 does that.

> Built `dfn-geos:1.1` before? The script says *"already exists — skipping the build"* and finishes in seconds. Nothing here is destructive, so it is safe to re-run any time.

Step 5 — Start the panel

```
python run_panel.py
```

You should see:

```
dfn-GEOS Workbench panel → http://localhost:8765
```

If instead you see `cannot listen on port 8765`, another program owns that port — use `python run_panel.py --port 9000` and open http://localhost:9000. **Do not** keep browsing 8765 in that case: you would be looking at the other program, not the panel.

Leave this window **open** — it is the server. Now open your browser (Chrome/Edge/Firefox) and go to:

http://localhost:8765

To stop the panel later, click in the PowerShell window and press **Ctrl+C**.

Step 6 — Settings (do this once)

If you used the fast track or step 4b, the script has already written these values — open **Settings** and just check that they match:

Field	Value
dfnWorks backend	`docker`
dfnWorks docker image	`ees16/dfnworks:latest`
dfnWorks repository	`C:\dfn-project\dfnWorks`
GEOS backend	`docker` (pre-configured by `-WithGeos`)
GEOS docker image	`dfn-geos:1.1` — or leave **blank**, the panel finds it automatically
GEOS executable	leave **empty** (the panel uses `/opt/geos/bin/geosx` inside the image)

There is nothing to change — click **Save** only if you edited something. The dashboard now shows the paths it will use.

> Did you skip step 4b? Then only the dfnWorks rows are needed: enter them exactly as above and leave the GEOS fields on *auto* — stage 3 will not run until the image is built (step 9 shows how).

Step 7 — Run your first case

1. Click **Quick process** (top bar).
2. Drag a **ZIP** onto the page — or click *pick files* and select your files.
3. Optionally type a case name.
4. Click **Process**.

The panel recognises what you gave it and runs the right steps:

Your input	What the panel does
`full_mesh.inp`, `params.txt`, `polys\`, `dfm_mesh\`	converts to a GEOS model
`dfn_driver.py`, `input_generator.dat`, `case_params.json`	generates the dfnWorks mesh, then converts
`*.xml` + `*.vtm` / `*.vtu`	runs GEOS on your own deck
nothing recognised	runs the whole pipeline with defaults

A normal case takes **2–10 minutes**. The page shows each stage turning green; the **Log** button next to a stage shows the full output.

The three stages, in one line each:

Stage	What it does	What it leaves behind
1 · dfnWorks mesh	builds the random fracture network and meshes it (dfnGen → LaGriT)	`output\full_mesh.inp`, `params.txt`, `polys\`, `dfm_mesh\`
2 · Convert → GEOS	translates those meshes into GEOS input	`geos_case\` (`.vtm`, `.vtu`, conformal deck) and `geos_case_edfm\geos_dfn_edfm.xml`
3 · Run GEOS	simulates flow in the rock + fractures, using the `dfn-geos:1.1` image from step 4b	`dfn_edfm.pvd` + one folder of `.vtu` per time step

> No input file yet? Use the example shipped with the project: open `C:\dfn-project\dfn-geos-workbench\poc\output`, select the files, and zip them — that is a real dfnWorks result you can convert immediately.

Step 8 — Collect the results

Open the case and click ⬇ **Download results (.zip)**, then unzip it.

```
geos_case\
  case.vtm                  conforming model: rock block + fracture surfaces
  main.vtu                  rock matrix (tetrahedra)
  fracture.vtu              explicit fracture surfaces (triangles + aperture)
  geos_dfn_fractures.xml    conformal GEOS input deck
geos_case_edfm\
  geos_dfn_edfm.xml         EDFM GEOS input deck — this is the one to run
dfn_edfm.pvd                the simulation time series (from stage 3)
dfn_edfm\0000N0\...         one folder per time step (matrix + fractures)
output\                     dfnWorks files (generated or uploaded)
```

Looking at the results in ParaView

Install **ParaView** from https://paraview.org (free). Then:

To see	Open this file	Then
The simulation over time	`dfn_edfm.pvd`	**Apply**, press ▶ to animate t = 0 … 50 000 s
Pressure in the rock	(tree) `matrix/Level0/RockMatrix`	colour by **`pressure`**
The fracture network	(tree) `matrix/Level0/Fracture`	colour by `pressure` or `elementAperture`; use *Surface With Edges* to see the 2-D elements
Just the geometry	`geos_case\case.vtm`	colour by `fracture_id` (fractures) or `attribute` (rock)

> Open the **`.pvd`**, not the individual `.vtu`: one `.vtu` is a single time step of a single region.

If the colour dropdown looks empty, toggle the Point/Cell-array button next to it — GEOS writes cell data.

Everything here (and the full list of files, plus velocity and CSV output) is explained in **`docs\USER_GUIDE.md`, section 10**.

Step 9 — How the GEOS image was built, and the alternatives

Stage *3 · Run GEOS* needs a GEOS binary, and GEOS only exists for Linux. That is exactly what the `dfn-geos:1.1` image from step 4b is: a Linux container with the compiled solver at `/opt/geos/bin/geosx`. When stage 3 runs, the panel starts that image and executes the solver inside it — you never touch Linux yourself.

How the image got there:

- **`-WithGeos` (what you did in step 4b)** — the setup script cloned the GEOS source, compiled it inside the `geosx/ubuntu24.04-gcc13` toolchain container, and saved the result as `dfn-geos:1.1`. Because the toolchain image already contains every third-party library, no `thirdPartyLibs` folder is ever needed on your PC; the GEOS source is used only for this one compile.

Alternatives, if you ever need them:

- **You skipped `-WithGeos`** — run `powershell -ExecutionPolicy Bypass -File .\setup_windows.ps1 -WithGeos` any time. Everything already on the PC is re-checked in seconds and only the GEOS build runs (~45–60 min). The panel then detects the image by itself.
- **Docker build by hand** — the validated command set is in `docs\INSTALL_windows.md`, section 5b.

- **WSL2 (easiest)** — install Ubuntu from the Microsoft Store and follow `docs\INSTALL_ubuntu.md` inside it; run the panel from there too.

Without the image, stage 3 fails fast with *"GEOS is not configured"* — that is expected and harmless: the mesh and decks are complete.

Step 10 — Next time you use it

Only two steps are needed after a restart:

```
cd C:\dfn-project\dfn-geos-workbench
start_windows.cmd
```

…or, the same thing by hand:

```
cd C:\dfn-project\dfn-geos-workbench
python run_panel.py
```

…then open http://localhost:8765. Docker Desktop must be running.

Troubleshooting

What you see	What to do
`'git' is not recognized`	Reinstall Git, then **close and reopen** PowerShell
`'python' is not recognized`	Reinstall Python with *Add python.exe to PATH* ticked
`'docker' is not recognized`, or *cannot connect to the Docker daemon*	Start Docker Desktop and wait for *Engine running*
`python run_panel.py` → *No module named numpy*	Run `python -m pip install numpy` again
Port 8765 already in use	`python run_panel.py --port 9000`, then open port 9000
Browser shows gibberish / binary instead of the panel	You are on a port owned by another program — start with `--port 9000` and open that
Stage 1: *no intersecting fractures*	Make fractures larger, or lower *Mesh h*
Stage 1: `MO DOES NOT EXIST: mo_poisson_pts`	Old LaGriT found — keep *dfnWorks backend* = `docker`
Stage 4b: the GEOS build seems stuck or takes longer than expected	It is not stuck: the submodule download moves several GB silently, then ~3000 C++ files compile for 30–60 minutes with no output. Leave the window alone — 45–60 minutes total is normal, longer on a slow disk
Stage 4b: *out of memory*, the build is killed, or *no space left on device*	The build needs about **8 GB** of free disk and memory for 4 parallel compiler jobs. Free disk space, stop other containers (`docker ps` , then `docker rm <name>`), and run the step 4b command again — it starts over cleanly
Stage 4b: docker error partway through, e.g. *cannot connect to the Docker daemon*	Docker Desktop was closed or restarted during the build. Open it, wait for *Engine running*, and run the step 4b command again
Stage 3: *GEOS executable is not configured*	The `dfn-geos:1.1` image does not exist yet (`docker images` must list it). Run the setup again with `-WithGeos` — it skips everything already done — or build by hand per `docs\INSTALL_windows.md` , section 5b
Case looks stuck at 0 %	Open **Log**; the first run pulls the image and can take minutes
Opening `dfn_edfm.pvd` shows **only XML text**	That is normal — it is data, not a picture. Open it in **ParaView** (step 8)
Files owned by `root` on Linux/WSL	`docker run --rm -v "$PWD":/w ees16/dfnworks:latest chown -R 1000:1000 /w`

One-page command summary

```
mkdir C:\dfn-project
cd C:\dfn-project
git clone https://github.com/sadafiiman/dfn-geos-workbench.git
cd C:\dfn-project\dfn-geos-workbench
powershell -ExecutionPolicy Bypass -File .\setup_windows.ps1 -WithGeos
```

That one command pulls the dfnWorks image, clones the GEOS source, builds the `dfn-geos:1.1` solver image (**~45–60 min, ~8 GB** — skipped automatically if the image already exists), writes the settings and starts the panel.

Settings (written for you by the script): dfnWorks backend `docker` · image `ees16/dfnworks:latest` · repository `C:\dfn-project\dfnWorks` · GEOS backend `docker` · image `dfn-geos:1.1` · GEOS executable empty.

Next time: `start_windows.cmd`, then http://localhost:8765.

Full reference: `docs\USER_GUIDE.md` (panel), `docs\INSTALL_windows.md` (platform + GEOS build), `report\feasibility_report.md` (technical results).